

Université de Montpellier
Faculté d'économie
DU Big data, data science et analyse des risques sous python

Guide de Référence du Projet Python : Rapport Détaillé sur le Plan de Travail

Babacar GUEYE

Amadou Sakho NDIAYE

Kobenan GBOKO

PLAN

- I. Enoncé du projet**
- II. Import Packages & Import Data**
- III. Prétraitement des données**
 - a. DB_sin**
 - b. DB_cnt**
 - c. DB_Telematics**
- IV. Visualisation**
- V. Feature Engineering**
- VI. Jointures**
 - a. Fusion_df1**
 - b. Fusion_df2**
- VII. Modélisation**
 - a. Driving Style**
 - b. Nb_Claim**
 - c. AMT_Claim**
- VIII. Tarification ou Classification (Clustering)**
- IX. Interface Shiny**

I. Enoncé du projet

Vous êtes un groupe de data scientist/analyst juniors travaillant pour une insurtech. Vous travaillez sur un projet d'exploitation de nouvelles bases de données d'assurance. Le seul outil à votre disposition est Python. Vous disposez de quelques semaines pour proposer une analyse aussi pertinente que possible des données. Vous rendez ensuite un rapport présentant toute l'étude à votre équipe et hiérarchie.

Le projet vise à exploiter le potentiel des bases de données d'assurance pour extraire des informations utiles qui aideront à comprendre les assurés dans la base et à améliorer les décisions stratégiques pour la tarification par exemple. Il sert également à mettre en œuvre nos connaissances et notre maîtrise des outils de traitement de données, d'économétrie et de machine learning en Python.

II. Import Packages & Import Data

a. Import Packages

Nous utilisons Python pour son efficacité et ses bibliothèques étendues comme Pandas, NumPy et Scikit-learn pour manipuler, traiter et analyser les données d'assurance. Nous aurons également besoin d'importer seaborn as sns pour la visualisation. Pour la modélisation, nous parlerons des packages utilisés dans la section dédiée.

b. Import Data

Les données sont importées à partir des sources suivantes :

- **DB_SIN**
- **DB_CNT**
- **DB_TELEMATICS**

Ces trois fichiers comportent l'ensemble des informations disponibles sur la base client.

III. Prétraitement des données

a. DB_SIN

DB_SIN contient 4337 observations et 3 variables.

Id_pol : l'identifiant du client assuré.

NB_claim : le nombre de Claim (réclamation après sinistre).

AMT_claim : le montant total reçu.

- Le montant moyen de AMT_Claim est de 3136.
- Le montant maximum est de 104074.
- La plupart des gens (75%) dépassent 3702.

Les étapes du traitement du fichier DB_SIN :

- Vérifier et traiter les valeurs manquantes.
- Chercher et éliminer les doublons.
- Mettre Nb Claim au bon format et en entier (Integer).
- Mettre AMT Claim au bon format et en flottant (float).
- Remplacer les valeurs NaN, ANN par 0.

b. DB_CNT

Le DataSet a la forme (100399, 12). 100399 lignes et 12 colonnes.

- Duration : Durée de la couverture d'assurance d'un contrat donné, en jours
- Insured.age : Âge du conducteur assuré, en années
- Insured.sexe : Sexe du conducteur assuré (Homme/Femme)
- Car.age : Âge du véhicule, en années
- Matrimonial : Etat civil (Célibataire/Marié)
- Car.use : Utilisation du véhicule : Privé, Trajet, Agriculteur, Commercial
- Credit.score : Pointage de crédit du conducteur assuré
- Region : Type de région où réside le conducteur : rurale, urbaine
- Annual.miles.drive : Miles annuels attendus à parcourir déclarés par le conducteur
- Years.noclaims : Nombre d'années sans aucune réclamation
- Territory : Localisation territoriale du véhicule

Les étapes du traitement du fichier DB_CNT :

- **Duration** est la période pendant laquelle le preneur d'assurance est assuré en jours, avec des valeurs en [22 ; 366].
- **Insured.age** est l'âge du conducteur assuré en années entières, avec des valeurs en [16,103].
- **Car.age** est l'âge du véhicule, avec des valeurs en [-2,20]. Les valeurs négatives sont rares mais possibles car l'achat d'un modèle plus récent peut se faire jusqu'à deux ans à l'avance.
- **Years.noclaims** est le nombre d'années sans aucune réclamation, avec des valeurs entre [0, 79] et toujours inférieur à l'âge assuré.
- **Territory** est le code de localisation territorial du véhicule, qui comporte 55 étiquettes en {11,12,13,...,91}.

Pour ces étapes, nous avons utiliser python en plus de la gestion des valeurs manquantes, les doublons et l'imputation de valeurs.

c. DB_TELEMATICS

DB_TELEMATICS contient 100332 lines.

Les étapes du traitement du fichier DB_CNT :

Nous voyons que la variable Id_pol est sous une forme différente (de la forme 'cnt_XXX') et que certaines colonnes sont des objets alors qu'elles devraient être des décimaux. Il y'a aussi des doublons à drop.

- Annual.pct.driven doit etre compris entre 0 et 1.1
- Pct.drive.mon, . . . , Pct.drive.sun leur somme doit faire 1. Donc => 100%.

IV. Visualisation

On cherche à visualiser les données afin de trouver des relations entre les données et surtout de mettre en avant les informations.

Pour cela, on effectue des boxplots, des histogrammes, une heatmap (pour voir la corrélation entre les variables) surtout après avoir fusionné nos bases.

Cette étape est cruciale car elle permet de voir l'impact et la logique entre le comportement des assurés et leur identité.

Quelques exemples :

La relation entre l'âge, le credit.score, l'âge du véhicule, son utilisation (car.use). La Région et l'Annual Miles. Entre autres.

V. Feature Engineering

En regardant les données dans TELEMATICS, on voit bien qu'il y'a de la redondance entre le 'percent.monday to Sunday', le kilométrage rush hour (pm et am), les niveaux d'accélération et de brake (freinage).

Feature Engineering :

To simplify the dataset and extract meaningful features, we can define the following types of variables through feature engineering:

1. **Demographic Features**:

- **Age Group**: Categorize insured driver's age into groups (e.g., young adult, middle-aged, senior).
- **Gender Binary**: Convert 'Insured.sex' into a binary variable (0 for male, 1 for female).

2. **Policy and Vehicle Features**:

- **Policy Duration Category**: Group 'Duration' into categories (e.g., short-term, medium-term, long-term).
- **Vehicle Age Group**: Categorize 'Car.age' into groups (e.g., new, moderately old, old).
- **Credit Score Group**: Bin 'Credit.score' into categories (e.g., poor, fair, good, excellent).
- **Car Use Category**: Create dummy variables for 'Car.use' (Private, Commute, Farmer, Commercial).
- **Region Type**: Convert 'Region' into binary variable (0 for rural, 1 for urban).

3. **Traditional Driving Behavior Features**:

- **Average Annual Miles Group**: Categorize 'Annual.miles.drive' into groups (e.g., low mileage, moderate mileage, high mileage).
- **Years Without Claims Group**: Categorize 'Years.noclaims' into groups (e.g., no claims, 1-2 years, 3-5 years, more than 5 years).
- **Territory Code**: Convert 'Territory' into binary variables (one-hot encoding) or drop the variable since we have the info in 'Region'.

4. **Telematics Driving Behavior Features**:

- **Driving Intensity**: Sum of 'Left.turn.intensityxx' and 'Right.turn.intensityxx' for overall turning intensity.
- **Weekly Driving Patterns**: Sum of 'Pct.drive.xxx' for overall weekly driving patterns.
- **Hourly Driving Patterns**: We will drop these variables since we believe most of the information is stored in Milages.
- **Rush Hour Driving**: Sum of 'Pct.drive.rushxx' for overall rush hour driving.
- **Safe Driver**: Composite score based on smooth driving behavior with low acceleration, braking, and turn intensity.
- **Aggressive Driver**: Composite score based on high acceleration, braking, and turn intensity.

5. **Response Variables**:

- **Claim Frequency**: Count the number of claims during the observation period ('Response NB Claim').
- **Claim Severity**: Aggregate amount of claims during the observation period ('AMT Claim').

By engineering these variables, we can simplify the dataset and create features that capture important aspects of driver behavior, vehicle characteristics, and insurance policy details, which can be used for modeling and analysis purposes.

Urban > Acceleration > Brake

Drive.Hour(Rural/urban)

Il nous est donc venu l'idée de créer une nouvelle variable qui synthétise le maximum d'information.

Cette nouvelle variable est le Driving Style. Nous l'avons défini ainsi :

```
# Calcule de la moyenne mean pour chaque variable
mean_light_acc = fusion_df1['Light_Accel_sum'].mean()
mean_intense_acc = fusion_df1['Intense_Accel_sum'].mean()
mean_light_brake = fusion_df1['Light_Brake_sum'].mean()
mean_intense_brake = fusion_df1['Intense_Brake_sum'].mean()
mean_light_turn = fusion_df1['Light_turn_sum'].mean()
mean_intense_turn = fusion_df1['Intense_turn_sum'].mean()
mean_sudden_turns = fusion_df1['Sudden_turns'].mean()
```

```
# Definition de la variable qui determine 'drive style'
```

```
def determine_drive_style(row):
```

```
    if (row['Light_Accel_sum'] <= mean_light_acc and
        row['Intense_Accel_sum'] <= mean_intense_acc and
        row['Light_Brake_sum'] <= mean_light_brake and
        row['Intense_Brake_sum'] <= mean_intense_brake and
        row['Light_turn_sum'] <= mean_light_turn and
        row['Intense_turn_sum'] <= mean_intense_turn and
        row['Sudden_turns'] <= mean_sudden_turns):
        return 'Safe'
```

```
    else:
```

```
        return 'Aggressive'
```

```
# Creation de la variable 'Drive_Style'
```

```
fusion_df1['Drive_Style'] = fusion_df1.apply(determine_drive_style, axis=1)
```

VI. Jointures

a. fusion_df1

fusion_df1 est la base obtenue après la fusion des deux plus grandes bases (DB_CNT et DB_TELEMATICS). Pourquoi ? Afin de garder une plus grande base, pour entraîner nos données pour la modélisation plus tard.

b. fusion_df2

fusion_df2 est la base obtenue après la fusion de *fusion_df1* et DB_SIN.

Elle est aussi grande grâce à la `left.join`

VII. Modélisation

Le Random Forest s'est avéré être l'algorithme le plus efficace sur nos données.

a. Driving Style

Nous avons utilisé la base *fusion_df1*, drop `id_pol` et Driving style sur le train set, créé au préalable, et on a testé plusieurs algorithmes de machine learning afin de voir le celui le plus efficace sur nos données de test. Le but de cette démarche est de pouvoir prédire en ayant de l'information sur l'assuré si celui-ci a une conduite '*safe*' ou une conduite '*aggressive*'.

Cela est un outil pour gérer le risque et donc un bon indicateur pour le pricing (tarification).

b. NB_Claim

Nous avons utilisé la base *fusion_df2*, cette fois-ci, on a ensuite drop `id_pol` et on a choisi de prendre les lignes avec NB_Claim (1,2,3) comme `train_set` et on a appliqué plusieurs algorithmes sur le reste comme `test_set` (une partie des données) et on a retenu le meilleur modèle sur l'ensemble des données afin de pouvoir prédire le NB_Claim de tous les assurés.

Pouvoir prédire le nombre de sinistre est un edge important pour l'assureur. Cela permet de quantifier le risque et d'adapter la tarification.

c. AMT_Claim

Nous avons tout simplement utilisé la base avec les NB_Claim prédits, et on a refait le meme processus que précédemment afin de prédire le AMT_Claim. Donc le montant reçu / réclamé.

On s'est rendu que le modèle mettait beaucoup trop de temps à calculer les sorties et surtout que l'accuracy tournait autour de 15%. On a donc décidé qu'il n'était pas judicieux d'essayer de predire cette valeur.

AMT_Claim est une valeur qu'il faut déterminer avec une tarification (pricing) tournant sur les caractéristiques du client (assuré).

VIII. Tarification

Avec toutes les informations rassemblées et surtout prédites concernant les assurés. Nous avons pensé à une stratégie de tarification basée sur plusieurs éléments (variables) pertinents permettant de gérer le risque.

Nous avons pensé à plusieurs manières de mettre une tarification. On a penser à une tarification basée sur l'expérience (NB_claim, years.noclaim, car.age), sur la classe de risque, sur le groupe (age,sexe,region).

Nous avons décidé finalement de créer une base avec (Age, sexe, NB_Claim, AMT_Claim, Total.miles.driven, Driving Style, Region, Car.age, Car.use, credit.score, Years.noclaim) et de faire 3 clusters avec K-means.

Chacun des **4 clusters** trouvé avec la methode Elbow (Eboulis) représente une tarification. Ce n'est pas la méthode la plus dynamique mais elle synthétise bien les données. Et le prix moyen couvre l'assureur (espérance mathématique).

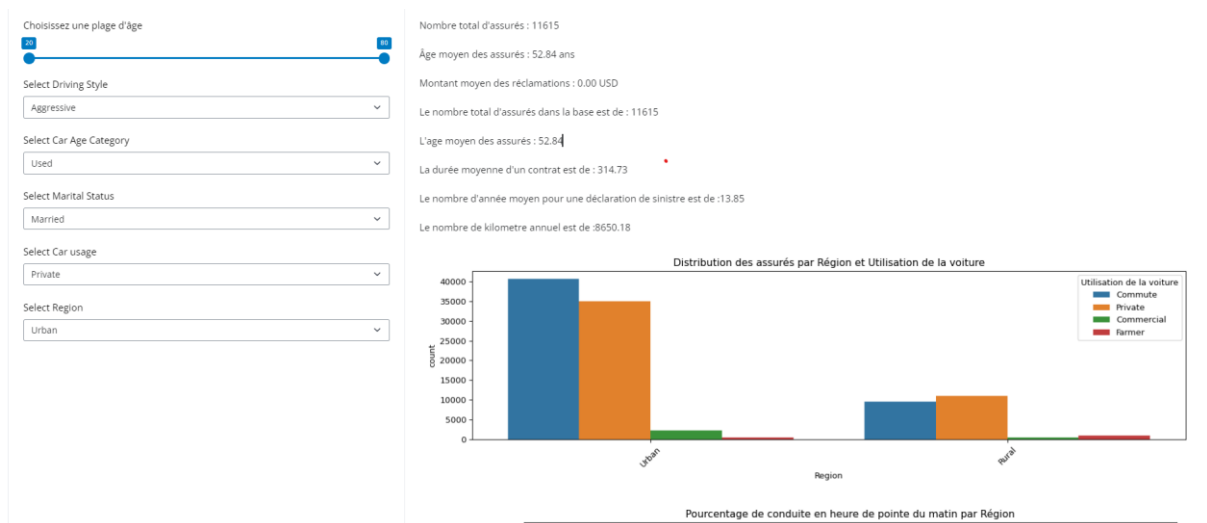
U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ
Region	Annual.miles.driven	Years.noclaim	Territory	Car_age_cat	Light_Accel_s	Intense_Accel	Intense_Brake	Light_tur_nu	Intense_tur_nu	Sudden_turns	Drive_Style	NB_Claim	AMT_Claim	Cluster	Tarification
Urban	6213	4	39	Used	82	0	77	0	5949	489	6438	Aggressive	1.0		1 Tariff 2
Urban	6213	13	13	Old	18	0	31	0	10	0	10	Safe	1.0		0 Tariff 3
Urban	12427	24	83	Used	8	0	23	0	0	0	0	Safe	1.0		1 Tariff 2
Rural	12427	11	69	New	47	0	81	0	2104	44	2148	Safe	1.0		1 Tariff 2
Rural	6213	0	90	Old	15	0	85	0	102	1	103	Safe	1.0		1 Tariff 2
Urban	4970	8	76	Used	129	0	191	4	1868	6	1874	Aggressive	1.0		1 Tariff 2
Rural	12427	24	90	Old	2	0	21	0	8	0	8	Safe	1.0		3 Tariff 4
Urban	12427	0	78	Old	75	0	216	1	35	0	35	Aggressive	1.0		1 Tariff 2
Rural	6213	49	24	Old	2	0	13	0	16	0	16	Safe	1.0		3 Tariff 4
Rural	9320	18	23	Used	17	0	31	0	206	4	210	Safe	1.0		1 Tariff 2
Urban	5592	0	39	New	34	2	51	2	257	6	263	Aggressive	1.0		1 Tariff 2
Rural	9320	10	43	Used	2	0	11	0	884	9	893	Safe	1.0		1 Tariff 2
Urban	12427	0	43	Used	35	0	191	1	170	12	182	Aggressive	1.0		0 Tariff 3
Rural	12427	12	18	New	9	0	41	0	20	0	20	Safe	1.0		1 Tariff 2
Urban	6213	41	75	Used	18	0	35	0	41	0	41	Safe	1.0		3 Tariff 4
Rural	9320	50	36	Used	2	0	30	0	2	0	2	Safe	1.0		3 Tariff 4
Urban	6213	9	74	Old	251	0	431	1	303	3	306	Aggressive	1.0		3 Tariff 4
Urban	6213	17	63	New	194	0	407	1	7034	458	7492	Aggressive	1.0		3 Tariff 4
Urban	12427	31	43	New	14	0	45	0	0	0	0	Safe	1.0		3 Tariff 4
Urban	12427	9	84	New	18	0	54	1	105	3	108	Aggressive	1.0		1 Tariff 2
Urban	12427	0	43	New	50	0	439	1	276	2	278	Aggressive	1.0		1 Tariff 2
Urban	6213	9	74	Old	183	0	50	0	62	0	62	Aggressive	1.0		3 Tariff 4
Urban	6213	12	18	Used	42	2	210	1	2000	45	2045	Aggressive	1.0	4546.0	1 Tariff 2
Urban	6213	29	77	Old	54	0	400	0	1694	3	1697	Aggressive	1.0		3 Tariff 4
Urban	12427	14	64	Used	4	0	60	0	1449	41	1490	Safe	1.0	4770.0	1 Tariff 2

IX. Interface Shiny

Une interface Shiny Python pour afficher des informations clés comme :

Le nombre total d'assurés, l'âge moyen, le montant moyen AMT_Claim, le kilométrage et les distributions graphiques des assurés (Region, Car.use), (Pct.drive.rush x Region).

Voici le Dashboard obtenu :



Le code est dans un fichier annexe :

Shiny_bg_Dashboard.py